

快速查找

我们小的时候都玩过一个游戏——你想我猜。就是指一个人心里想一个数字，然后告诉你一个范围，要求你用最少的时间猜出来。每当你猜一个数的之后，如果不正确就会告诉你猜的数大了还是小了。请你设计一种策略更快的猜出正确的数字。

有的同学就是从起点一直往终点猜，有的同学就是随便乱猜一个数组，无论是以上哪一种猜的方法，都很难高效的得到答案，那有没有一种高效的办法呢？答案是有的。

每次猜测起点和终点的一半，如果比实际值大，就猜测起点到中点的中间值，如果比实际值小，就猜测中点到终点的中间值，这样下去，肯定会更快得到答案。

二分思想

二分查找

其实生活中还有很多实用二分查找的案例。比如电路故障查找，求高次方程的零点，根据重量查找质量不合格的物品等等。这些都需要用二分的思想来解决问题。

对于一个长度为 n 的有序序列。如果依次查找，找到正确结果的时间复杂度为 $O(n)$ ，但是如果用二分的方法查找。每次排除一半的可能性，在剩下的一半里面找，这样每次查找，范围变为以前的一半。时间复杂度为 $O(\log n)$

每次确定起点和终点后，判断中间点是否是要找的值，如果是，就输出答案，如果不是，判断大了还是小了，然后重新确定起点和终点，再判断中间点是否是要找的值。。。如此循环下去。可以用循环或者递归来实现。

查找给定的值

已知一个长度为 n 的有序递增序列，现给定 m 次查询，求每次查询的数在序列中的位置，如果查询的数在序列中，输出位置，如果不在序列中，输出“No Number”。

输入：

4

1 3 5 7

3

7 3 0

输出：

4

2

No Number

数据范围： $n \leq 100000, m \leq 100000$

查找给定的值

对于一个长度为8的序列：1 3 5 7 9 11 13 15。

如果要查找数字1，第一次比较中点的值 $a[(1+8)/2]=7$ ，大于1，第二次比较中点的值 $a[(1+3)/2]=3$ ，第三次比较中点的值 $a[(1+1)/2]=1$ ，等于我们要查找的值。

如果要查找数字2，前两次查找的方法都和前面第一样，当第三次查找的时候， $a[(1+1)/2]=1$ ，小于我们要查找的值，第四次查找范围是 $[1+1, 1]$ ， $left > right$ ，区间没有值，所以我们要查找的数不存在

所以当找到我们需要查找的值(存在)或者当查找区间内无值(不存在)的时候，表示二分查找结束

快速排序

我们以前学习的排序，速度都特别慢，比如冒泡排序，每次比较之后，总让后一个数大于前一个数，当第一趟确定一个最大值的时候，第二次继续让后一个数大于前一个数，但是实际上在第一趟交换的时候，很多地方已经交换过了，所以浪费了很多时间。

而我们后面将要学习的几种效率更高的排序，都用到了二分的思想，所以自然就能把时间复杂度优化下来。

快速排序：通过一趟排序将要排序的数据分割成独立的两部分，其中一部分的所有数据都比另外一部分的所有数据都要小，然后再按此方法对这两部分数据分别进行快速排序

快速排序



↑
i: 指向左
端点指针

↑
j: 指向右
端点指针

第一步：以当前区间的左端点(6)为基准值（排序之后左边的数都小于等于基准值，右边的数都大于等于基准值）

第二步：设立两个指针，一个指向左端点，一个指向右端点。先j指针向左移动，找到一个<基准值(6)的时候就停下来(5)，然后i指针向右移动，找到一个>基准值(6)的时候就停下来(7)，说明5和7的位置都是不合理的，所以交换，交换后两个数的位置都合理了，然后继续上述操作，直到 $i=j$ 。



① ② $i==j$ ② ①

快速排序

6	1	2	5	4	3	9	7	10	8
---	---	---	---	---	---	---	---	----	---

$i=j$

第三步：此时我们就会发现 $i=j$ 这个位置左边的数都小于等于基准值，右边的数都大于等于基准值，但是 $i=j$ 这个位置的数是不合理的，所以交换基准值和该位置的值，`swap(6, 3)`

3	1	2	5	4	6	9	7	10	8
---	---	---	---	---	---	---	---	----	---

经过一轮排序之后，我们确定了基准值的真实位置，并且把一个无序序列分成了两个独立的部分，基准值左边的数 $\leq$ 基准值右边的数，我们再分别把这两部分按照上面的操作来做，就可以很快的得到一个有序的序列。

1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	----

快速排序

为什么要先j左移再i右移：

假设是i先右移，j后左移，那么说明i停留的位置一定是 $\geq$ 基准值，但是j停留的位置不一定是 $\leq$ 基准值的，假设在右边找不到一个值 $<$ 基准值，那么j就会一直左移直到 $i=j$ ，那么此时相遇的点的数 $\geq$ 基准值的，此时执行交换操作，那么基准值左边的数就不是完全小于等于他了，就会出错。

6	1	2	7	9
i				j

如果j先左移，到达2的时候停下来，然后i右移，也到达2，然后交换6和2的值，是正确的。

如果i先右移，那么到达7的时候就会停下来，然后j左移，当到达7的时候， $i=j$ ，也会停下来，然后交换6和7的值，明显是错误的

快速排序

快速排序的时间复杂度不是固定的，因为每次不能严格的把一个序列分成两个长度相近的子序列，快速排序的平均时间复杂度是 $O(n\log n)$ ，最坏时间复杂度是 $O(n^2)$ ，比如当数列整体有序的时候，每次确定的值都是端点值，所以当在一个序列中大部分数据是重复的时候，快速排序的时间复杂度也远远高于 $O(n\log n)$ 。

如果一个序列总体有序，可以用随机一个序列中的值做基准值来解决，但是竞赛考试中不可以用`time`库，所以不可以生成随机数，我们可以直接假定一个基准值，然后和第一个数交换。比如中位数啥的，或者随便想一个值 k ，注意 k 一定要在该范围内。但是还是没有办法解决多重重复数据的问题。

快速排序

```
void quicksort(int left, int right)
```

```
{
```

```
    //交换第一个值和该区间内任意一个值
```

```
    确定第一个数为基准值;
```

```
    确定左右端点指针
```

```
    while(i!=j)
```

```
    {
```

```
        while(a[j]>=基准值&& i<j)            j--;
```

```
        while(a[i]<=基准值&& i<j)            i++;
```

```
        如果i, j不相等;        swap(a[i], a[j]);
```

```
    }
```

```
    swap(基准值, a[j]);
```

```
    左边部分继续排序;
```

```
    右边部分继续排序;
```

```
}
```

一元三次方程求解

有形如： $ax^3+bx^2+cx+d=0$ 这样的的一个一元三次方程。给出该方程中各项的系数(a, b, c, d 均为实数)，并约定该方程存在三个不同实根(根的范围在 -100 至 100 之间)，且根与根之差的绝对值 ≥ 1 。要求由小到大依次在同一行输出这三个实根(根与根之间留有空格)，并精确到小数点后2位。

提示：记方程 $f(x)=0$ ，若存在2个数 x_1 和 x_2 ，且 $x_1 < x_2$ ， $f(x_1)*f(x_2) < 0$ ，则在 (x_1, x_2) 之间一定有一个根。

输入：

1 -5 -4 20

输出：

-2.00 2.00 5.00

一元三次方程求解

对于一个方程而言，如果 $f(m) * f(n) < 0$ ，那么说明 $[m, n]$ 区间内至少有一个零点，改题已经明确说明根与根的绝对值只差 ≥ 1 ，那就说明对于 $f(m) * f(m+1) < 0$ ，说明 $[m, m+1]$ 内肯定有一个零点，否则没有零点。

所以我们可以依次遍历 $[-100, 100]$ 区间内的 $[m, m+1]$ ，如果满足条件就说明 $[m, m+1]$ 内肯定有一个解，同样的，如果 $f(m) == 0$ 就直接有解了。

当确定区间之后，我们可以用二分的方法缩小区间范围，如果 $f(mid) * f(left) < 0$ 说明在左区间内，反之就在右区间内，要求精确到小数点后两位，所以当 $right - left < 0.001$ 的时候就结束，随便输出 $left$ 或者 $right$ 。

注意数据范围都要用double